

Case Study:- Retail Sales and Customer Insights: A Comprehensive SQL Analysis

1. Introduction

1.1 Objective

The objective of this case study is to analyze customer purchasing behavior, product sales trends, and order performance using a structured SQL database. By leveraging MySQL, we aim to extract meaningful insights that can help businesses optimize inventory management, improve customer engagement, and maximize revenue.

1.2 Dataset Overview

This case study is based on three datasets:

1. **Customers (200 records)** - Contains customer details such as name, email, phone number, city, and country.
2. **Products (50 records)** - Includes product details such as name, category, price, and stock quantity.
3. **Orders (800 records)** - Stores order transactions, including customer purchases, product details, quantity, and order amount.

These datasets are linked through:

- **CustomerID** (to associate orders with customers)
 - **ProductID** (to associate orders with products)
-

2. Database Schema Design

2.1 Customers Table

Column Name	Data Type	Description
CustomerID	INT	Unique customer identifier
CustomerName	VARCHAR(255)	Customer's full name
Email	VARCHAR(255)	Customer's email address
Phone	VARCHAR(50)	Contact number
Address	TEXT	Customer's address
City	VARCHAR(100)	Customer's city
Country	VARCHAR(100)	Customer's country

2.2 Products Table

Column Name	Data Type	Description
ProductID	INT (PK)	Unique product identifier
ProductName	VARCHAR(255)	Name of the product
Category	VARCHAR(100)	Product category (e.g., Electronics, Fashion)
Price	DECIMAL(10,2)	Product price
StockQuantity	INT	Number of items available

2.3 Orders Table

Column Name	Data Type	Description
OrderID	INT (PK)	Unique order identifier
CustomerID	INT (FK)	Customer who placed the order
ProductID	INT (FK)	Ordered product
Quantity	INT	Number of items ordered
OrderAmount	DECIMAL(10,2)	Total order amount
OrderDate	DATE	Date when order was placed

3. Data Preprocessing

Before analysis, we need to ensure data consistency:

- Check for **null or missing values** and handle them appropriately.
 - Validate **data types** to match the expected format.
 - Ensure **foreign key integrity** between tables.
 - Remove duplicate records if any exist.
 - Normalize data to improve efficiency.
-

4. Case Study Questions

SQL Case Study Questions

BASIC LEVEL

-- 1)Fetch all products available in the store.

```
SELECT  
    ProductName  
FROM  
    Products;
```

-- 2)Count the total number of customers.

```
SELECT  
    COUNT(CustomerID) AS Total_Customers  
FROM  
    Customers;
```

-- 3)Count the total number of products available.

```
SELECT  
    COUNT(ProductID) AS Total_Products  
FROM  
    Products;
```

-- 4)Find the total revenue from orders.

```
SELECT  
    SUM(OrderAmount) AS Total_Revenue  
FROM  
    Orders;
```

-- 5)Retrieve customer details who live in "East James".

SELECT

*

FROM

Customers

WHERE

City = 'East James';

-- 6)Get product details where the price is greater than \$500.

SELECT

*

FROM

Products

WHERE

Price > 500;

-- 7)Find distinct cities from where customers are registered.

SELECT DISTINCT

City

FROM

Customers;

-- 8)Find the maximum, minimum, and average price of products.

SELECT

MAX(Price) AS Maximum_Price,

MIN(Price) AS Minimum_Price,

AVG(Price) AS Average_Price

FROM

Products;

-- 9)Retrieve the most expensive product.

SELECT

ProductName AS Most_Expensive_Product

FROM

Products

WHERE

Price = (SELECT

MAX(Price)

FROM

Products);

-- 10)Retrieve the cheapest product.

```
SELECT
    ProductName AS Cheapest_Product
FROM
    Products
WHERE
    Price = (SELECT
        MIN(Price)
        FROM
            Products);
```

-- 11)Fetch all orders placed in the last 30 days.

```
SELECT
    *
FROM
    Orders
WHERE
    OrderDate >= NOW() - INTERVAL 30 DAY;
```

-- 12)Get the total number of customers from each country.

```
SELECT
    Country, COUNT(*) AS Total_Customers
FROM
    Customers
GROUP BY Country;
```

-- 13)Find total revenue per month for 2024.

SELECT

MONTH(OrderDate) AS Months, SUM(OrderAmount) AS Revenue

FROM

Orders

WHERE YEAR(OrderDate)=2024

GROUP BY Months

ORDER BY Months;

-- 14)List orders details sorted by highest order amount.

SELECT

*

FROM

Orders

ORDER BY OrderAmount DESC;

-- 15)Retrieve the order details where quantity is more than 2.

SELECT

*

FROM

Orders

WHERE

Quantity > 2;

INTERMEDIATE LEVEL

-- 16)List all customers who placed at least one order.

SELECT

C.CustomerName, O.Quantity

FROM

Customers C

JOIN

Orders O USING (CustomerID)

WHERE

O.Quantity >= 1;

-- 17)Find customers who have never placed an order.

SELECT

C.CustomerName

FROM

Customers C

LEFT JOIN

Orders O USING (CustomerID)

WHERE

O.CustomerID IS NULL;

-- 18)Find the total amount spent by each customer.

SELECT

C.CustomerName, SUM(O.OrderAmount) AS Total_Spent

FROM

Customers C

JOIN

Orders O USING (CustomerID)

GROUP BY CustomerName;

-- 19)Find the top 5 customers based on total spending.

SELECT

C.CustomerName AS Top_5_Customers,

SUM(O.OrderAmount) AS Total_Spent

FROM

Customers C

JOIN

Orders O USING (CustomerID)

GROUP BY C.CustomerName

ORDER BY Total_Spent DESC

LIMIT 5;

-- 20)Find the top 3 most ordered products.

```
SELECT
    P.ProductName AS Most_Ordered_Products,
    SUM(O.Quantity) AS Quantities
FROM
    Products P
    JOIN
    Orders O USING(ProductID)
GROUP BY P.ProductName
ORDER BY Quantities DESC
LIMIT 3;
```

-- 21)Find the least ordered product.

```
SELECT
    P.ProductName, COUNT(O.ProductID) AS Number_Of_Orders
FROM
    Products P
    JOIN
    Orders O USING (ProductID)
GROUP BY P.ProductName
ORDER BY Number_Of_Orders
LIMIT 1;
```

-- 22)Retrieve the total revenue per product category.

SELECT

P.Category, SUM(O.OrderAmount) AS Total_Revenue

FROM

Products P

JOIN

Orders O USING (ProductID)

GROUP BY P.Category;

-- 23)Retrieve the total number of orders per customer.

SELECT

C.CustomerName, COUNT(O.CustomerID) AS Total_Orders

FROM

Customers C

JOIN

Orders O USING (CustomerID)

GROUP BY C.CustomerName;

-- 24)Find the total revenue generated per city.

SELECT

C.City, SUM(O.OrderAmount) AS Total_Revenue

FROM

Customers C

JOIN

Orders O USING (CustomerID)

GROUP BY C.City;

-- 25)Find products that were never ordered.

SELECT

P.ProductName

FROM

Products P

LEFT JOIN

Orders O USING (ProductID)

WHERE

O.ProductID IS NULL;

-- 26)Retrieve the total stock by category available in the store.

SELECT

Category, SUM(StockQuantity) AS Available_Stock

FROM

Products

GROUP BY Category;

-- 27)Find products with less than 20 stock quantity.

SELECT

ProductName, StockQuantity

FROM

Products

WHERE

StockQuantity < 20;

-- 28)Find orders where the order amount is greater than the average order amount.

SELECT

 OrderID,OrderAmount

FROM

 Orders

WHERE

 OrderAmount > (SELECT

 AVG(OrderAmount)

 FROM

 Orders);

-- 29)Retrieve all orders placed in the last 6 months with total amount spent per customer.

SELECT

 C.CustomerName, SUM(O.OrderAmount) AS Total_Spent

FROM

 Customers C

 JOIN

 Orders O USING (CustomerID)

WHERE

 O.OrderDate >= NOW() - INTERVAL 6 MONTH

GROUP BY C.CustomerName;

-- 30)Find the number of orders placed per month in the last year.

```
SELECT
    MONTH(OrderDate) AS Month_Number, COUNT(OrderID) AS Total_Orders
FROM
    Orders
WHERE
    YEAR(OrderDate) = YEAR(NOW())-1
GROUP BY Month_Number
ORDER BY Month_Number;
```

ADVANCED LEVEL

-- 31)Find the top 3 customers who placed the highest number of orders.

```
SELECT
    Ranking, CustomerID, CustomerName, Orders_Count
FROM
    (SELECT
        O.CustomerID, C.CustomerName, COUNT(O.CustomerID) AS
Orders_Count,
        RANK() OVER(ORDER BY COUNT(O.CustomerID) DESC) AS Ranking
    FROM
        Customers C
        JOIN
        Orders O USING(CustomerID)
    GROUP BY C.CustomerName, O.CustomerID
    ) AS SUB
WHERE Ranking <= 3;
```

-- 32)Find the month with the highest total sales.

```
SELECT
    Ranking,Month_Number,Total_Sales
FROM
    (
        SELECT
            MONTH(OrderDate) AS Month_Number,SUM(OrderAmount) AS
Total_Sales,
            RANK()OVER(ORDER BY SUM(OrderAmount) DESC)AS Ranking
        FROM
            Orders
        GROUP BY Month_Number
    )AS Sub1
WHERE Ranking=1;
```

-- 33)Retrieve customers who placed orders in every month of the last year.

```
SELECT
    O.CustomerID, C.CustomerName
FROM
    Orders O
    JOIN
        Customers C USING (CustomerID)
WHERE
    YEAR(O.OrderDate) = YEAR(NOW()) - 1
GROUP BY O.CustomerID , C.CustomerName
HAVING COUNT(DISTINCT MONTH(O.OrderDate)) = 12;
```


-- 34)Find the product that generated the highest revenue.

```
SELECT
    Ranking,ProductName,Total_Revenue
FROM
    (
        SELECT
            P.ProductName,SUM(O.OrderAmount) AS Total_Revenue,
            RANK()OVER(ORDER BY SUM(O.OrderAmount) DESC)AS Ranking
        FROM
            Products P JOIN Orders O USING(ProductID)
        GROUP BY P.ProductName
    )AS SUB1
WHERE Ranking=1;
```

-- 35)Retrieve customers who ordered at least 3 different products.

```
SELECT
    O.CustomerID, C.CustomerName
FROM
    Customers C
    JOIN
        Orders O USING (CustomerID)
GROUP BY O.CustomerID , C.CustomerName
HAVING COUNT(O.ProductID) >= 3;
```

-- 36) Find the first and last purchase date for each customer.

```
SELECT
    O.CustomerID,
    C.CustomerName,
    MIN(O.OrderDate) AS First_Purchase_Date,
    MAX(O.OrderDate) AS Last_Purchase_Date
FROM
    Customers C
    JOIN
    Orders O USING (CustomerID)
GROUP BY O.CustomerID , C.CustomerName;
```

-- 37) Find the percentage of revenue contributed by each product category.

```
SELECT
    P.Category,
    ROUND(SUM(O.OrderAmount) * 100 / (SELECT
        SUM(OrderAmount)
    FROM
        Orders),
    2) AS Percentage
FROM
    Products P
    JOIN
    Orders O USING (ProductID)
GROUP BY P.Category;
```

-- 38)List products along with their sales count.

SELECT

P.ProductName, COUNT(O.ProductID) AS Sales_Count

FROM

Products P

JOIN

Orders O USING (ProductID)

GROUP BY P.ProductName;

-- 39)Identify the most frequently ordered category.

SELECT

Category AS Most_Frequently_Ordered_Category,Sales_Count

FROM

(

SELECT

P.Category,COUNT(O.ProductID) AS Sales_Count,

RANK()OVER(ORDER BY COUNT(O.ProductID) DESC) AS Ranking

FROM

Products P

JOIN

Orders O USING (ProductID)

GROUP BY P.Category

)AS sub1

WHERE Ranking=1;

-- 40)Retrieve customers whose total spending exceeds the average spending of all customers.

SELECT

CustomerID, CustomerName, Total_Spending

FROM

(SELECT

O.CustomerID,

C.CustomerName,

SUM(O.OrderAmount) AS Total_Spending

FROM

Customers C

JOIN Orders O USING (CustomerID)

GROUP BY O.CustomerID , C.CustomerName) AS CustomerSpending

WHERE

Total_Spending > (SELECT

AVG(Total_Spending)

FROM

(SELECT

SUM(OrderAmount) AS Total_Spending

FROM

Orders

GROUP BY CustomerID) AS AvgSpending);

5. Business Insights & Analysis

Based on the queries, we can derive the following insights:

- **Customer Segmentation:** Identify high-value customers based on spending.
 - **Product Performance:** Determine which products sell the most and which are underperforming.
 - **Sales Trends:** Find peak seasons or months for sales activity.
 - **Inventory Management:** Identify products with low stock that need restocking.
 - **Revenue Optimization:** Understand which product categories generate the most revenue.
-

6. Conclusion & Recommendations

6.1 Summary

This case study provides a structured approach to analyzing customer behavior, sales trends, and product performance using MySQL. By leveraging relational database concepts, businesses can optimize decision-making processes and improve profitability.

6.2 Recommendations

- Focus marketing efforts on high-value customers.
 - Increase stock for best-selling products.
 - Offer discounts on slow-moving products to boost sales.
 - Improve engagement with customers who have not placed orders in a long time.
 - Monitor seasonal trends and optimize inventory accordingly.
-

7. Future Scope

This study can be extended by:

- Implementing **predictive analytics** using machine learning.
 - Integrating **real-time dashboards** with Power BI or Tableau.
 - Expanding the dataset with customer demographics for deeper analysis.
-